

Relatório L1: Criptografia e One-Time Pad

19 de agosto de 2026

1 Identificação do Estudante

Nome: Cristiano Gregory Monfrim Camandaroba

Matrícula: 2026687700

Disciplina: Cibersegurança

2 Descrição da Implementação e Decisões Adotadas

A cifra One-Time Pad (OTP) foi implementada em Python por meio de um script unificado, com o objetivo de garantir a manipulação segura e precisa dos dados em nível de bytes. Para a geração das chaves criptográficas, utilizou-se a biblioteca nativa `secrets`, especificamente a função `secrets.token_bytes()`, que fornece aleatoriedade adequada para fins criptográficos, ao contrário do módulo `random`. Todas as operações foram realizadas com objetos do tipo `bytes`. No caso de textos em ASCII ou UTF-8, efetuou-se previamente sua codificação para bytes. A função central de cifração e decifração percorre as sequências de bytes e aplica o operador XOR bit a bit, retornando o resultado também como um objeto do tipo `bytes`.

Durante a manipulação de arquivos em disco, adotou-se estritamente a abertura nos modos binários de leitura e gravação (`'rb'` e `'wb'`). Essa abordagem de baixo nível evitou que o sistema operacional corrompesse os dados ao tentar interpretar os bytes cifrados como caracteres de texto ou quebras de linha. Para atestar a integridade de todo o processo, implementou-se a biblioteca `hashlib` para gerar resumos criptográficos em SHA-256, fornecendo uma prova matemática e irrefutável de que o arquivo decifrado era um clone exato do original. A apresentação dos resultados no terminal foi formatada com o método hexadecimal, garantindo uma exibição visualmente limpa e padronizada, sem poluir a tela com caracteres de escape do sistema. Por fim, como medida de mitigação de riscos e boas práticas de segurança, o script foi programado para remover ativamente a chave do diretório local ao término das operações, prevenindo o armazenamento simultâneo da chave e dos dados cifrados.

3 Evidências dos Testes (Partes A e B)

3.1 Parte A - Testes de Codificação

Os testes foram realizados considerando três formatos de entrada distintos: texto em ASCII, texto com caracteres especiais (UTF-8) e uma sequência binária sintética. Em

todos os casos, a mensagem original coincidiu perfeitamente com a recuperada. Abaixo está o trecho da execução demonstrando a formatação hexadecimal limpa:

```
PS D:\ic\Itens\facul_no_geral\UFMG\Meistrado\Ciberseguranca > & C:/Users/crist/AppData/Local/Programs/Python/Python310/python.exe "d:/ic\Itens\facul_no_geral\UFMG\Meistrado\Ciberseguranca/Laboratorio_1/otp.py"
=====
PARTE A - TESTES DE CODIFICAÇÃO
=====

[ TESTE 1: ASCII ]
--- Apresentação em Hexadecimal ---
Mensagem Original: 45737461286520756d61286d656e736167656d28646520746573746520717565206e616f28636f6e74656d286e656e68756d286163656e746f286f752063617261637465726520657378656369616c2e28656617363f2864612867616d612e
Chave Utilizada: f8e6845a28d6a3b89a23f45f869c1689e5a19a2ae7649ac69d1f3753dec9538601d377e917e80e77e927035791b3a17275b1fcb5987857c0156f716348a2384c41cfad6a7b693e9448d4143808f4bb643f18bf84e93b2bc69974cedd118b
Mensagem Cifrada: b595f836eb383c5f742af2895eb289f13f7482ca1369d8ca287101d9fbc36a66fb218c974876083c4a2339f4d0c90718919d0651623af3580044323c3a2d22bbc8181e495be738b1775169895446971cc67861b4fa7b913afb67eas
Verificação: SUCESSO! Mensagem ASCII recuperada perfeitamente.

[ TESTE 2: UTF-8 ]
--- Apresentação em Hexadecimal ---
Mensagem Original: 412863726978746f67726166696128646f286c61626f726174c3b372696f2870726f7465676572c3a1286e6f73736f73286461646f7328636f6e666964656e6369616973212856617363f2864612867616d6121
Chave Utilizada: 7285a3884f6e4f39aa8847cb87a6f5a6cdf349787b51528c36f9396bc22da745809f91cdd82d4d77fe9407dc1d4e11f1b589ad7f131ff5e341fedb6baca74b78826cd8d223859e60cd4c743b83ed4911ba
Mensagem Cifrada: 33a5cf8a261e3b56cdfa26adec7d5c2a2d32511193e286d423a8a19ab428735f2f8e5a8bae7a614dec92e12b2a78e6c3b3c8bb3986edf3d5b718bdfdea7ad46147a7e0cf874592a856fed28151be48c24709b
Verificação: SUCESSO! Mensagem UTF-8 recuperada perfeitamente.

[ TESTE 3: Binário Sintético ]
--- Apresentação em Hexadecimal ---
Mensagem Original: 7a86e3315d03037ee9e6283ba7845c08506ebdeeb8e3f46e97e56f79b4aebf095875dd526f229600f18e30323938a91c30
Chave Utilizada: f9c21238ba1c772e150e7e2f8e25c4d892f772f2d975d42999db46e5a84d9588a98a7e5a2c77cc00347693f1a92737943e74
Mensagem Cifrada: 834fcb0af41742db0e71cac045d811817f19923c27beb0f6a5189ad1f93b37a0daeb87f698ee9634871dc19b1e0f3d2244
Verificação: SUCESSO! Sequência binária recuperada perfeitamente.
```

Figura 1: Print do terminal da parte A após rodar o código

A execução da Parte A demonstrou a eficácia do algoritmo OTP aplicado a diferentes tipos de codificação. Conforme o trecho de execução extraído do terminal, é possível observar a representação puramente hexadecimal das matrizes de bytes. O sistema validou de forma automatizada que a mensagem decifrada é perfeitamente idêntica à original. Isso evidencia que a implementação atua de forma correta e agnóstica em nível de baixo nível, aplicando a operação lógica XOR sem perda de dados, seja manipulando texto simples (ASCII) ou caracteres com mapeamento de múltiplos bytes (UTF-8).

A diferença entre o número de caracteres de uma string e o seu tamanho em bytes ocorre porque o padrão UTF-8 utiliza um sistema de codificação de tamanho variável. Isso significa que ele não gasta a mesma quantidade de espaço na memória para representar todas as letras do nosso alfabeto ou símbolos digitais.

Caracteres básicos da tabela original ASCII, como letras de "A" a "Z" sem acentuação, números e marcações de pontuação simples, são muito padronizados e ocupam exatamente um byte cada. No entanto, quando utilizamos caracteres mais complexos ou regionais, como letras acentuadas ("á", "ç", "ã"), símbolos matemáticos, emojis ou até alfabetos inteiros de outros idiomas (como o mandarim ou o cirílico), o computador precisa de um intervalo numérico maior para conseguir representar todas essas opções únicas.

Para resolver isso, o UTF-8 passa a alocar de dois a quatro bytes para renderizar um único caractere visual. É por esse motivo que, na prática, uma palavra de quatro letras sem acentos (como "pato") terá exatamente quatro bytes de tamanho, enquanto uma palavra também de quatro letras, mas com acentuação (como "ação"), ocupará seis bytes de espaço na memória, já que o "ç" e o "ã" exigirão dois bytes cada um para serem processados corretamente pelo sistema.

3.2 Parte B - Cifrando um Arquivo Sintético

Na Parte B, a avaliação ocorreu sobre a manipulação de um arquivo físico em disco (registros_rede_sinteticos.csv). As evidências de execução confirmam que a integridade estrutural foi estritamente preservada durante o processo de leitura e gravação binária, visto que os arquivos original, cifrado e recuperado mantiveram o tamanho exato de 897 bytes, sem adição de metadados indesejados. Para a validação definitiva, utilizou-se a biblioteca `hashlib` para calcular e comparar os resumos criptográficos SHA-256 do arquivo original e do arquivo recuperado, conforme sugerido pelo roteiro. Os hashes idên-

ticos provam matematicamente que nenhum bit foi corrompido durante o ciclo completo de cifração e decifração.

```
=====
PARTE B - CIFRANDO UM ARQUIVO SINTÉTICO
=====
--- Registro de Tamanhos ---
Original:  897 bytes
Cifrado:   897 bytes
Recuperado: 897 bytes

--- Verificação de Integridade (SHA-256) ---
Hash Original:  0b9ff7ac7c9d17b4541755f377b013a11dcf0d5bcb3a10536631a62ab6bfef53
Hash Recuperado: 0b9ff7ac7c9d17b4541755f377b013a11dcf0d5bcb3a10536631a62ab6bfef53

SUCESSO: Os hashes coincidem! O arquivo foi recuperado com perfeição.

Aviso: O arquivo 'chave.key' foi removido do diretório por segurança.
```

Figura 2: Print do terminal da parte B após rodar o código

Redução de segurança ao armazenar a chave localmente:

Armazenar a chave criptográfica e o texto cifrado no mesmo local compromete totalmente a segurança do sistema, pois a força da criptografia moderna depende exclusivamente de manter a chave em segredo absoluto. Em caso de uma invasão, se um atacante conseguir acessar o servidor ou o diretório onde os arquivos estão guardados, ele obterá simultaneamente os dados protegidos e a ferramenta exata necessária para abri-los, criando o que chamamos de falha de ponto único. Como a decifração em métodos simétricos (como o One-Time Pad implementado) é uma operação instantânea e determinística quando se tem a chave correta, o esforço de criptografar os dados torna-se completamente inútil. Na prática, é o equivalente a guardar um documento ultrassecreto em um cofre de aço impenetrável e deixar a chave colada na porta com uma fita adesiva: o cofre continua sendo tecnicamente forte, mas a proteção real foi anulada pela má gestão do acesso.

4 Demonstração Algébrica e Experimental (Parte C)

A segurança do OTP depende estritamente do uso de uma chave única. Para demonstrar essa vulnerabilidade, as mensagens M_1 e M_2 foram extraídas diretamente do arquivo `mensagens_teste.txt` em modo binário. Em seguida, reutilizou-se deliberadamente a mesma chave K para cifrar essas duas mensagens distintas, obtendo-se:

$$C_1 = M_1 \oplus K$$

$$C_2 = M_2 \oplus K$$

Ao calcularmos o XOR entre os dois textos cifrados ($C_1 \oplus C_2$), a propriedade de cancelamento do XOR anula a chave K :

$$C_1 \oplus C_2 = (M_1 \oplus K) \oplus (M_2 \oplus K) = M_1 \oplus M_2$$

Experimento Prático: Conforme demonstrado na execução do código, o atacante, conhecendo M_1 e C_1 , recupera facilmente a chave fazendo $K = M_1 \oplus C_1$. Com a chave

roubada em mãos, ele aplicou $M_2 = C_2 \oplus K$ e conseguiu decifrar a segunda mensagem secreta.

```
=====
PARTE C - ATAQUE DE REUSO DE CHAVE
=====
Tamanho de M1 lido: 61 bytes
Tamanho de M2 lido: 61 bytes

--- Iniciando o Ataque ---
PASSO 1 [SUCESSO]: Confirmado! C1 ⊕ C2 é exatamente igual a M1 ⊕ M2.
PASSO 2 [SUCESSO]: O atacante recuperou a chave original K perfeitamente!
PASSO 3 [SUCESSO]: O atacante decifrou M2 com sucesso usando a chave roubada.

--- Teste com Chaves Independentes (O jeito certo) ---
PASSO 5 [SUCESSO]: A invasão falhou! Como as chaves eram independentes, o hacker obteve apenas lixo numérico ao tentar decifrar M2.
```

Figura 3: Print do terminal da parte C após rodar o código

Violação da condição essencial da OTP: O experimento realizado viola a condição mais fundamental e inegociável da cifra One-Time Pad, que é a exigência do uso único e exclusivo da chave criptográfica. O próprio nome do algoritmo dita que a chave deve ser utilizada apenas uma vez ("*one-time*") e imediatamente descartada. Essa regra é o que garante o seu sigilo perfeito, pois assegura que o texto cifrado seja sempre matematicamente indistinguível de um ruído aleatório. Ao reutilizarmos deliberadamente a mesma chave para cifrar duas mensagens distintas, criamos uma grave vulnerabilidade algébrica. Como demonstrado, a propriedade do XOR faz com que a chave duplicada se anule ao cruzar as mensagens cifradas, expondo as mensagens originais em texto claro. A partir desse momento, a premissa da segurança inquebrável deixa de existir, pois o sistema permite que um invasor recupere a chave original e decifre todas as comunicações caso descubra o conteúdo de apenas uma das mensagens.

5 Respostas da Parte D

1. A cifra One-Time Pad oferece confidencialidade perfeita porque, sem a posse da chave correta, o texto cifrado é matematicamente indistinguível de um ruído aleatório, impossibilitando a leitura do conteúdo original por terceiros. No entanto, ela não garante integridade, autenticidade ou não repúdio porque a operação matemática XOR não adiciona nenhuma assinatura digital ou código de verificação estrutural (como algoritmos de hash) que permita ao destinatário confirmar quem enviou a mensagem ou se ela foi corrompida em trânsito. Um exemplo claro dessa vulnerabilidade é o ataque de alteração de bits (bit-flipping). Se um invasor interceptar um texto cifrado contendo uma transferência bancária e souber a posição exata do byte que representa o valor, ele pode aplicar um novo XOR com um dado calculado por ele mesmo nesse local. Quando a instituição financeira decifrar o pacote, o sistema revelará um montante adulterado e não haverá qualquer indício algorítmico de que o arquivo sofreu essa manipulação maliciosa.
2. A principal dificuldade logística na distribuição de chaves OTP reside na exigência matemática irreduzível de que a chave deve ter exatamente o mesmo tamanho do volume de dados a ser protegido. Caso dois pontos de uma rede corporativa precisem trocar um terabyte de informações confidenciais diariamente, será mandatório gerar e entregar ao destino um terabyte de chaves a cada dia. Transferir esse material criptográfico colossal de forma totalmente segura exigiria canais de comunicação com

largura de banda impraticável ou uma operação física complexa e excessivamente custosa, como o transporte regular de discos rígidos através de carros-fortes. Essa exigência de paridade de tamanho inviabiliza completamente o uso do One-Time Pad para redes modernas e contínuas de alto tráfego.

3. O ciclo de vida de uma chave OTP apresenta riscos críticos em todas as suas fases operacionais, iniciando pela etapa de geração, onde o uso de funções pseudoaleatórias comuns de software pode resultar em sequências previsíveis, anulando a segurança perfeita do sistema. Na fase de transporte, o maior perigo recai sobre a interceptação da chave caso ela seja transmitida pelo mesmo meio inseguro por onde trafegarão os arquivos cifrados. Durante o armazenamento e a rotina de cópias de segurança (backups), o risco principal é o arquivamento indevido das chaves no mesmo servidor dos dados ou em mídias sem controle de acesso, o que cria um ponto único de falha desastroso em caso de invasões ou vazamentos por membros internos da organização. Finalmente, na fase de descarte, a exclusão lógica padrão fornecida pelos sistemas operacionais não destrói as informações fisicamente do disco, permitindo que invasores utilizem ferramentas de recuperação forense para resgatar chaves antigas e ler comunicações já finalizadas.
4. A diferença conceitual fundamental entre o One-Time Pad e uma cifra de fluxo moderna reside na origem e na proporção do material criptográfico empregado na operação XOR. O algoritmo OTP demanda uma chave verdadeiramente aleatória e tão extensa quanto a mensagem em si, o que garante a inviolabilidade absoluta, mas gera gargalos intransponíveis de armazenamento e transporte. Em contrapartida, as cifras de fluxo modernas operam a partir de uma chave inicial estática e extremamente pequena (geralmente de 256 bits), que funciona apenas como uma semente inicial. Essa pequena chave alimenta um algoritmo matemático determinístico complexo que atua como um motor, expandindo a semente infinitamente para gerar o fluxo pseudoaleatório necessário para cifrar os dados. Com isso, os algoritmos modernos abdicam da segurança matemática perfeita e absoluta em favor de uma segurança computacional altamente prática, mantendo os dados protegidos contra ameaças contemporâneas sem a necessidade de distribuir gigabytes de chaves.
5. Para assegurar a integridade do ciclo de vida das chaves em um ambiente profissional, propõe-se a aplicação de quatro controles essenciais e rigorosos. Primeiramente, a etapa de geração deve ser realizada exclusivamente por Geradores de Números Aleatórios Verdadeiros em hardware (TRNGs), que utilizam processos físicos, como ruído térmico, para produzir entropia real e imprevisível. Em segundo lugar, o armazenamento seguro exige o uso de Módulos de Segurança de Hardware (HSMs), que são equipamentos blindados e programados para destruir os dados internamente caso detectem violação física ou lógica. O terceiro controle refere-se à distribuição, que deve ocorrer estritamente por canais alternativos (out-of-band), garantindo que a chave e o texto cifrado jamais percorram a mesma infraestrutura de rede. Por último, o descarte deve adotar técnicas de apagamento criptográfico irrecuperável, consistindo na sobrescrita contínua dos setores do disco rígido com dados aleatórios, ou até mesmo na destruição física completa da mídia de armazenamento para eliminar permanentemente os vestígios da chave.

6 Conclusão

A execução deste laboratório permitiu responder de forma clara e fundamentada à questão norteadora sobre a viabilidade e as limitações do One-Time Pad (OTP) no contexto real da segurança da informação. Os resultados práticos obtidos nas primeiras etapas do experimento comprovaram a premissa teórica de que o OTP oferece confidencialidade absoluta. A aplicação rigorosa da operação XOR em nível de bytes garantiu a cifração e a recuperação exata de arquivos e textos, preservando a integridade estrutural dos dados de forma agnóstica ao seu formato. No entanto, a simulação de ataque conduzida na Parte C evidenciou a extrema fragilidade do sistema diante de falhas operacionais humanas ou sistêmicas. O experimento provou matematicamente que o mero reuso de uma chave anula por completo a proteção criptográfica, expondo o conteúdo original e validando a regra irredutível de que a chave deve ser estritamente de uso único.

Ao extrapolarmos esses resultados práticos para a análise de redes modernas na Parte D, conclui-se que o verdadeiro desafio da criptografia não reside apenas na formulação de algoritmos inquebráveis, mas na engenharia logística de sua aplicação. Apesar de sua invulnerabilidade matemática inquestionável, o OTP apresenta gargalos insustentáveis para o tráfego de dados contemporâneo, exigindo a geração, o transporte seguro e a destruição de chaves com o mesmo tamanho massivo das informações protegidas. Além disso, a ausência de mecanismos nativos para validar a integridade dos pacotes deixa o sistema vulnerável a adulterações de bits (bit-flipping). Portanto, os resultados do laboratório demonstram que a criptografia moderna precisou evoluir de forma pragmática, abdicando do sigilo teórico perfeito do OTP em favor de cifras algorítmicas (como as cifras de fluxo modernas) que, embora ofereçam "apenas" segurança computacional, resolvem de maneira viável e robusta os desafios críticos de integridade e do ciclo de vida das chaves.

7 Referências

- STALLINGS, William. *Criptografia e Segurança de Redes*.
- Documentação oficial do Python: Módulo `secrets`.